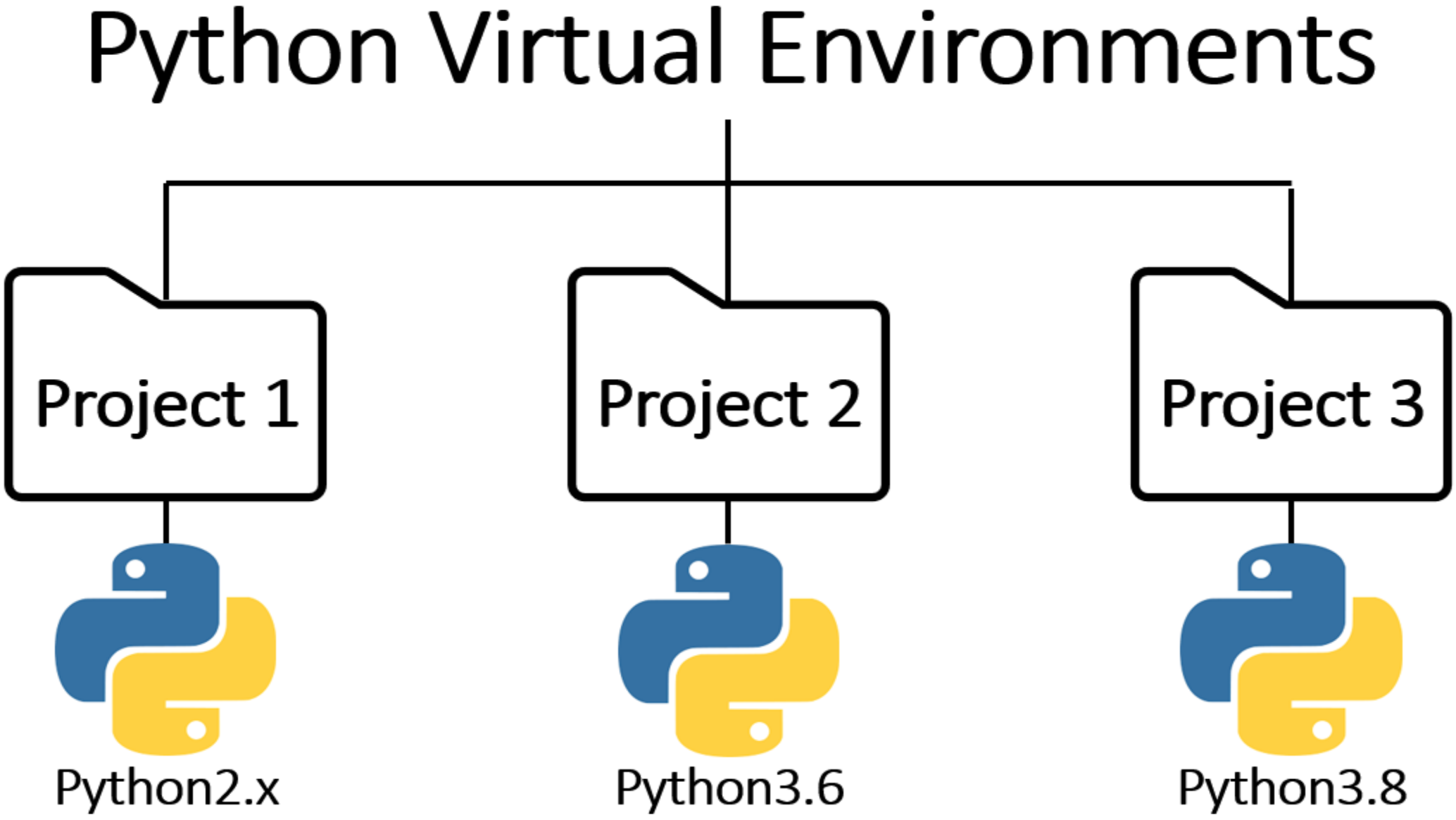
Basistechnieken en dependency management in Python



Na vandaag…

Weet je…

Welke verschillende Python-datastructuren er zijn en hoe deze overeenkomen en verschillen van elkaar

Wat het backwards-compatibilityprobleem is van Python

De twee belangrijkste pandas-datastructuren en hun structureringswijze.

Verschillende programmeertechnieken per wijzigingsactie

HC + WC

Kan je…

Programmeertechnieken op basisniveau uitvoeren in Python

Een virtual environment voor Python in VSCode maken met behulp van poetry

Data uit externe bestanden converteren naar pandas-datastructuren.

Rijen en kolommen in DataFrames wijzigen

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Taal en tools

Python

- Anno 2026 de meest gebruikte scripttaal voor Data Science.
- Relatief simpele syntax: print(“Hello World”). Klassen zijn mogelijk, maar niet noodzakelijk.
- Python is een scripttaal: coderegels worden direct uitgevoerd, zonder ze eerst in machinecode om te zetten.

Jupyter Notebook

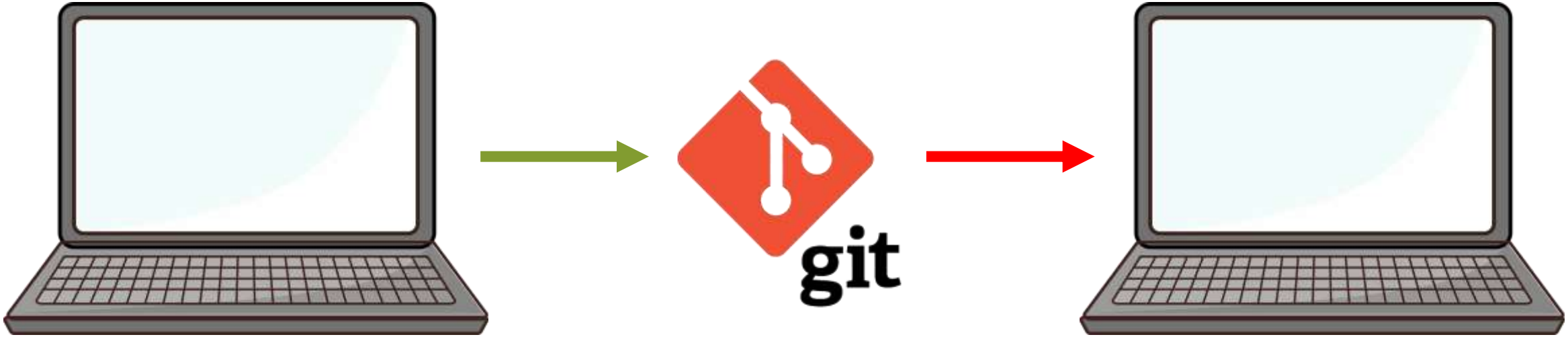
- Veelgebruikte Python-ontwikkelomgeving.
- Cellulaire structuur: opbouw uit code- en markdowncellen. Code kan stapsgewijs doorlopen worden zonder breakpoints te plaatsen.
- Bestandsextensie: .ipynb

Visual Studio Code

- Veelgebruikte IDE (ook toegepast in SEM3).
- Goede integratie met Jupyter Notebook (met de installeerbare “Jupyter Notebook Extension”).
- Goede integratie met git.

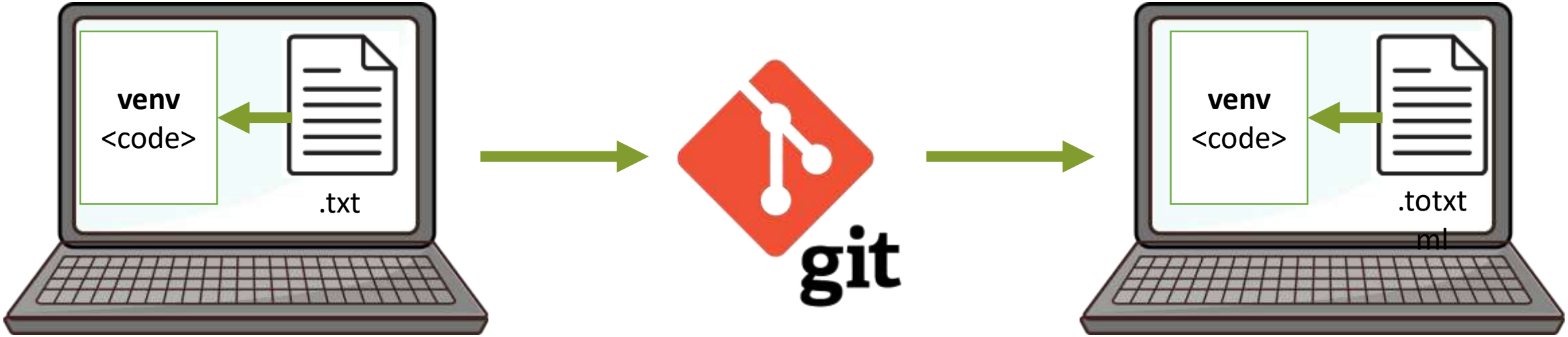
Backwards compatibility: risico

Pythonrisico: tegenvallende backwards compatibility.
Code die met de ene versie is geschreven, wordt niet automatisch ondersteund door de andere versie!
Dit geldt zowel voor de Python- als voor de package-versie(s).



Backwards compatibility: risico

Pythonrisico: tegenvallende backwards compatibility.
Code die met de ene versie is geschreven, wordt niet automatisch ondersteund door de andere versie!
Oplossing: een ‘virtual environment’ (‘venv’) creëren, waarin de juiste dependency-versies worden ingelezen vanuit een requirementsbestand



- In meerdere projecten kun je nu specifieke dependency-versies gebruiken.
- Je isoleert je project, waardoor andere mappen en bestanden op je computer het niet kunnen beschadigen.

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

4 verschillende virtual-environmenttools

De klassieke manier

Package-administratie: .txt

- Geen extra installaties en minimale configuratie
- Minder flexibel: geen ondersteuning van meerdere Pythonversies

virtualenv

Package-administratie: .txt

- Ondersteunt oudere Pythonversies
- Meer functionaliteit dan de klassieke manier.
- Extra installatie nodig

poetry

Package-administratie: .toml

- Alles-in-éénoplossing: lost problemen met traditionele tools op.
- Is ingewikkelder om op te zetten voor eenvoudige projecten.

pdm

Package-administratie: .toml

- Gebruiksgemak met automatische omgevingsbeheer
- Nieuw en minder bekend

Backwards compatibility & Basistechnieken

Bouw en gebruik van virtual environments - Handleiding

Van Java naar Python - Cheatsheet

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Jelle van Barneveld

HBO-ICT
DE HAAGSE HOGESCHOOL

Verzamelingen in Python

Datastructuur	Beschrijving	Kenmerkend(e) symbo(o)l(en)	Codevoorbeeld

List ('list')	Vergelijkbaar met Java-arrays. Op basis van indexnummers kunnen lijstelementen aangeroepen én aangepast worden.	Vierkantshaken bij het aanmaken én aanroepen:
		[en]
			`mijn_lijst = [1, 2, 3]`
			`print(mijn_lijst[0])`
			
			`mijn_lijst[0] = 2`
			`print(mijn_lijst[0])`

Tupel ('tuple')	Vergelijkbaar met lists en Java-arrays, maar aanpassingen zijn nu niet mogelijk! De rode regel in het codevoorbeeld resulteert daarom in een error.	Ronde haken bij het aanmaken:
		(en)
		
		Vierkantshaken bij het aanroepen:
		[en]
			`mijn_tupel = (1, 2, 3)`
			`print(mijn_tupel[0])`
			
			`mijn_tupel[0] = 2`
			`print(mijn_tupel[0])`

Dictionary ('dict')	Hierin gebruik je geen indexen om bij de 'values' te komen, maar zelfbedachte waarden ('keys'). Elke key wordt dus gekoppeld aan één waarde.	Accolades bij het aanmaken:
		{ en }
		
		Dubbele punten bij het onderscheiden van keys en values
		
		Vierkantshaken bij het aanroepen:
		[en]
			`mijn_dict = {`
			`'plaats' : 'Leiden',`
			`'inwoners' : 121562`
			`}`
			
			`print(mijn_dict['inwoners'])`
			
			`mijn_dict['inwoners'] += 1`
			`print(mijn_dict['inwoners'])`

Ook te lezen in "Van Java naar Python – Cheatsheet"

Dataframes uit de pandas-package

Een dataframe is een representatie van een tabel in de pandas-package in Python.
Data wordt op **twee** manieren gestructureerd: in **rijen** en **kolommen**. Dit is dus **2D-informatie**.
Let op! Dit kan alleen met **gestructureerde data:** data die “netjes” gestructureerd is in tabelvorm.

Excel

	A	B	C	D	E	F
1	nr	naam	woonplaa	baas	functie	uurloon
2	11	Piet	Leiden	15	PR	100
3	12	Elske	Leiden	15	PR	110
4	13	Jan	Lisse	15	SA	120
5	14	Chris	Berkel	NULL	PL	130
6	15	Peter	Leiden	14	PL	140
7	16	Kees	NULL	15	NULL	100

```
import pandas as pd
medewerker = pd.read_excel("DB_exports.xlsx", sheet_name = "Medewerker")
medewerker
```

=>

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100

Je moet óók de package openpyxl installeren!

Opbouw DataFrames

Een dataframe bestaat uit meerdere “pandas Series”: zij slaan 1D-data op.
Elke “pandas Series” correspondeert met één kolom uit het overkoepelende DataFrame

medewerker (variabele): pd.DataFrame

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100

=>

nr: pd.Series

```
0    11
1    12
2    13
3    14
4    15
5    16
Name: nr, dtype: int64
```

naam: pd.Series

```
0     Piet
1    Elske
2      Jan
3    Chris
4    Peter
5     Kees
Name: naam, dtype: object
```

Elke serie bestaat uit…

- Een naam
- Een datatype (int, float, bool, object, enzovoort)
- Een index (rood gemarkeerd): een label dat meegegeven wordt om elke rij uniek identificeerbaar te maken. **Welke kolom zou hier nog meer voor gebruikt kunnen worden?**

Combinaties met andere datastructuren

Vanuit een lijst kan je een serie maken. Vanuit een dictionary kan je een serie en een DataFrame maken.

```python
mijn_lijst = ['A', 'B', 'C', 'D', 'E']
pd.Series(mijn_lijst)
```

```
0    A
1    B
2    C
3    D
4    E
dtype: object
```

Let op: het datatype ‘str’ is gelijk aan ‘object’ in pandas!

```python
mijn_dictionary = {'Den Haag' : 'De Haagse Hogeschool', 'Leiden' : 'Hogeschool Leiden'}
pd.Series(mijn_dictionary)
```

✓ 0.0s

```
Den Haag    De Haagse Hogeschool
Leiden        Hogeschool Leiden
dtype: object
```

Elke waarde in de dict bestaat uit slechts één string!
Elke key in de dict wordt een index in de serie

```python
mijn_tweede_dictionary = {'Land' : ['Nederland', 'Marokko', 'Japan'], 'Werelddeel' : ['Europa', 'Afrika', 'Azie']}
pd.DataFrame(mijn_tweede_dictionary)
```

✓ 0.0s

	Land	Werelddeel
0	Nederland	Europa
1	Marokko	Afrika
2	Japan	Azie

Elke waarde in de dict bestaat uit een lijst met meerdere strings
Elke key in de dict wordt een kolom. De indexen in de serie worden automatisch gemaakt.

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Benodigde wijzigingsacties

Soort doel	Doel	Techniek
Data toevoegen	Rijen toevoegen	Dictionary met nieuwe waarden maken, converteren en toevoegen
	Kolommen toevoegen	Reeds bestaande kolommen combineren
Data wijzigen	Waarden in rijen wijzigen	Kolomoverschrijving
		.replace()
	Kolomnamen wijzigen	.rename(columns = <dict>)
Data verwijderen	Rijen verwijderen op basis van index	.drop(axis = 0)
	Kolommen verwijderen op basis van naam	.drop(axis = 1)

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL

Rijen toevoegen

Medewerkers ‘Henk’ en ‘Marleen’ moeten nieuw toegevoegd worden als rij.

Stap 1:

Een nieuwe dictionary maken

```
nieuwe_medewerkers_dictionary = {
    'nr':[17, 18],
    'naam':['Henk', 'Marleen'],
    'woonplaats':['Voorschoten', 'Amsterdam'],
    'baas' : [15, 15],
    'functie' : ['SA', 'SO'],
    'uurloon' : [120, 130]
}

nieuwe_medewerkers_dictionary
```

✓ 0.0s

```
{'nr': [17, 18],
 'naam': ['Henk', 'Marleen'],
 'woonplaats': ['Voorschoten', 'Amsterdam'],
 'baas': [15, 15],
 'functie': ['SA', 'SO'],
 'uurloon': [120, 130]}
```

Remember! Om een dictionary naar een DataFrame om te zetten, moeten per kolom alle waarden in een lijst staan!

Stap 2:

Converteren naar een nieuw DataFrame en deze met concat() toevoegen aan het originele DataFrame

```
pd.concat([medewerker, pd.DataFrame(nieuwe_medewerkers_dictionary)], ignore_index=True)
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

ignore_index = True betekent dat de index in het DataFrame automatisch optelt (naar “6” en “7” in dit geval).

Jelle van Barneveld

HBO-ICT

DE HAAGSE HOGESCHOOL

Kolommen toevoegen

Nieuwe kolommen kan je toevoegen door combinaties van reeds bestaande kolommen te maken.

```
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

Kolom “afkomst” toegevoegd

```
medewerker['afkomst'] = medewerker['naam'] + ' komt uit ' + medewerker['woonplaats']
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon	afkomst
0	11	Piet	Leiden	15.0	PR	100	Piet komt uit Leiden
1	12	Elske	Leiden	15.0	PR	110	Elske komt uit Leiden
2	13	Jan	Lisse	15.0	SA	120	Jan komt uit Lisse
3	14	Chris	Berkel	NaN	PL	130	Chris komt uit Berkel
4	15	Peter	Leiden	14.0	PL	140	Peter komt uit Leiden
5	16	Kees	NaN	15.0	NaN	100	NaN
6	17	Henk	Voorschoten	15.0	SA	120	Henk komt uit Voorschoten
7	18	Marleen	Amsterdam	15.0	SO	130	Marleen komt uit Amsterdam

Jelle van Barneveld

HBO-ICT

DE HAAGSE HOGESCHOOL

Waarden in rijen wijzigen met kolomoverschrijving

```
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

Alle uurlonen +10%

```
medewerker['uurloon'] *= 1.10
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	110.0
1	12	Elske	Leiden	15.0	PR	121.0
2	13	Jan	Lisse	15.0	SA	132.0
3	14	Chris	Berkel	NaN	PL	143.0
4	15	Peter	Leiden	14.0	PL	154.0
5	16	Kees	NaN	15.0	NaN	110.0
6	17	Henk	Voorschoten	15.0	SA	132.0
7	18	Marleen	Amsterdam	15.0	SO	143.0

Jelle van Barneveld

HBO-ICT

DE HAAGSE HOGESCHOOL

Waarden in rijen wijzigen met .replace()

```
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

‘Leiden’ wordt telkens gewijzigd naar ‘Leeuwarden’

```
medewerker = medewerker.replace('Leiden', 'Leeuwarden')
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leeuwarden	15.0	PR	100
1	12	Elske	Leeuwarden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leeuwarden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

Kolomnamen wijzigen

Wijzigingen…

- De kolomnaam ‘baas’ is ‘leidinggevende’ geworden.
- De kolomnaam ‘uurloon’ is ‘salaris’ geworden.

Er wordt dus een dictionary gemaakt, waarin de keys de oude kolomnamen weergeven en de values de nieuwe.

```
medewerker = medewerker.rename(columns = {'baas' : 'leidinggevende', 'uurloon' : 'salaris'})
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	leidinggevende	functie	salaris
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

Jelle van Barneveld

Rijen verwijderen met behulp van index

Met .drop() in combinatie met ‘axis = 0’ kunnen we kolommen verwijderen.
Deze specificatie van de axis-parameter mag je hier ook weglaten, 0 is namelijk de standaardwaarde.

```
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

De medewerker op index 6 wordt verwijderd.

```
medewerker = medewerker.drop(6, axis = 0)
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
7	18	Marleen	Amsterdam	15.0	SO	130

Kolommen verwijderen met behulp van kolomnaam

Met .drop() in combinatie met ‘axis = 1’ kunnen we kolommen verwijderen.

	nr	naam	woonplaats	baas	functie	uurloon	afkomst
0	11	Piet	Leiden	15.0	PR	100	Piet komt uit Leiden
1	12	Elske	Leiden	15.0	PR	110	Elske komt uit Leiden
2	13	Jan	Lisse	15.0	SA	120	Jan komt uit Lisse
3	14	Chris	Berkel	NaN	PL	130	Chris komt uit Berkel
4	15	Peter	Leiden	14.0	PL	140	Peter komt uit Leiden
5	16	Kees	NaN	15.0	NaN	100	NaN
6	17	Henk	Voorschoten	15.0	SA	120	Henk komt uit Voorschoten
7	18	Marleen	Amsterdam	15.0	SO	130	Marleen komt uit Amsterdam

```
medewerker = medewerker.drop('afkomst', axis = 1)
medewerker
```

✓ 0.0s

	nr	naam	woonplaats	baas	functie	uurloon
0	11	Piet	Leiden	15.0	PR	100
1	12	Elske	Leiden	15.0	PR	110
2	13	Jan	Lisse	15.0	SA	120
3	14	Chris	Berkel	NaN	PL	130
4	15	Peter	Leiden	14.0	PL	140
5	16	Kees	NaN	15.0	NaN	100
6	17	Henk	Voorschoten	15.0	SA	120
7	18	Marleen	Amsterdam	15.0	SO	130

Met bovenstaande code wordt de kolom ‘afkomst’ verwijderd.

Na vandaag…

Weet je…

Welke verschillende Python-datastructuren er zijn en hoe deze overeenkomen en verschillen van elkaar

Wat het backwards-compatibilityprobleem is van Python

De twee belangrijkste pandas-datastructuren en hun structureringswijze.

Verschillende programmeertechnieken per wijzigingsactie

HC + WC

Kan je…

Programmeertechnieken op basisniveau uitvoeren in Python

Een virtual environment voor Python in VSCode maken met behulp van poetry

Data uit externe bestanden converteren naar pandas-datastructuren.

Rijen en kolommen in DataFrames wijzigen

Jelle van Barneveld
HBO-ICT
DE HAAGSE HOGESCHOOL